ESCUELA DE INGENIERÍA DE FUENLABRADA

GRADO EN INGENIERÍA EN
TECNOLOGÍAS DE LA TELECOMUNICACIÓN

TRABAJO FIN DE GRADO

SISTEMA DE REPORTE SEGURO FRENTE A ROOTKITS
DE ESPACIO DE USUARIO

Autor: Noel Rodríguez Pérez

Tutor: Gorka Guardiola Múzquiz

Curso académico 2025-2026

Este trabajo se distribuye bajo los términos de la licencia internacional CC BY-NC-ND International License (Creative Commons Licencia Atribución/Reconocimiento-NoComercial-SinDerivados 4.0 International). Usted es libre de *compartir*: copiar y redistribuir el material en cualquier medio o formato. La licenciante no puede revocar estas libertades en tanto usted siga los términos de la licencia:

- *Atribución.* Usted debe dar crédito de manera adecuada, brindar un enlace a la licencia, e indicar si se han realizado cambios. Puede hacerlo en cualquier forma razonable, pero no de forma tal que sugiera que usted o su uso tienen el apoyo de la licenciante.
- *No Comercial.* Usted no puede hacer uso del material con fines comerciales.
- *Sin Derivadas.* Si remezcla, transforma o crea a partir del material, no podrá distribuir el material modificado.

  No hay restricciones adicionales — No puede aplicar términos legales ni medidas tecnológicas que restrinjan legalmente a otras a hacer cualquier uso permitido por la licencia.

No tiene que cumplir con la licencia para elementos del materiale en el dominio público o cuando su uso esté permitido por una excepción o limitación aplicable. No se dan garantías. La licencia podría no darle todos los permisos que necesita para el uso que tenga previsto. Por ejemplo, otros derechos como publicidad, privacidad, o derechos morales pueden limitar la forma en que utilice el material.

Documento de **Noel Rodríguez Pérez**.

Agradecimientos

En primer lugar, me gustaría agradecer a mi tutor, **Gorka Guardiola Múzquiz**, por su constante apoyo, paciencia y dedicación durante el desarrollo de este proyecto. Todo el conocimiento que me ha transmitido ha sido fundamental no solo para este trabajo, sino para saber el interés real que tengo por este campo de la Ciberseguridad. Gracias por enseñarme tanto.

A todos los docentes de mi carrera, por dotarme de conocimientos que me permiten ahora cerrar este ciclo y empezar mi carrera profesional de la mejor manera posible.

También, a todas las personas de mi entorno, por haberme acompañado en todo este proceso y haber disfrutado de su compañía y apoyo todo este tiempo.

Quiero agradecer especialmente a mi pareja, por su apoyo incondicional en mis decisiones y la gran ayuda que me ha aportado en mi desarrollo como persona. A mis padres y a mi hermana, por darme la educación y apoyo que me han hecho llegar a donde estoy actualmente. Y a mi grupo de la universidad, por haber hecho este ciclo mucho más ameno y haberme hecho disfrutar del proceso, incluso en momentos difíciles.

Muchas gracias por todo.

A mi familia, pareja y amigos;
mi mayor apoyo en toda esta etapa.

Madrid, 14 de febrero de 2026
Noel Rodríguez Pérez

Resumen

La seguridad en los sistemas modernos es un área en constante evolución, donde la integridad de los datos y el control de su funcionamiento juegan un papel fundamental. Esto es debido a la presencia de amenazas y agentes externos que trabajan con el fin malicioso de comprometer dicha información, haciendo necesario el desarrollo de mecanismos de protección cada vez más robustos.

En este ámbito, una de las amenazas más críticas para los sistemas actualmente son los *rootkits*, software malicioso diseñado para ocultarse en el núcleo del sistema (*kernel*) y controlar su funcionamiento. Al infiltrarse en los niveles más profundos, este *malware* adquiere permisos de administrador, permitiéndole controlar la ejecución de procesos de forma arbitraria y manipular respuestas e información del sistema operativo, impidiendo obtener una visión verídica del estado real de los archivos y procesos del sistema. Esto provoca una falta de integridad en la monitorización y control del mismo, donde por defecto se deposita una confianza implícita en una fuente de datos sensibles que puede estar comprometida ante dicho tipo de amenaza.

Para abordar esta problemática, en este proyecto se ha diseñado e implementado un *Módulo del Kernel* (*LKM*, *Loadable Kernel Module*) en un sistema *Linux* con el objetivo de protegerlo ante este tipo de amenazas. Esta solución introduce un mecanismo de autenticación criptográfica basado en HMAC (*Hash-based Message Authentication Code*) del contenido proporcionado por el *kernel*, mediante una clave simétrica secreta que solo es conocida por el administrador, garantizando la integridad de la información ofrecida.

Los experimentos realizados aproximan un entorno real de operación donde la información sensible es solicitada desde el *espacio de usuario* hacia el *espacio del kernel*. Con este fin, se define el contenido sensible del sistema a autenticar mediante un listado de todos los procesos en ejecución y sus respectivos metadatos, generado en el propio *Módulo del Kernel*, sirviendo como ejemplo de información crítica que puede ser

manipulada por un *rootkit*. Sobre este contenido, se calcula el HMAC con la respectiva clave secreta y se adjunta a la información proporcionada al usuario, de forma que este pueda verificar su integridad y autenticidad.

Para validar esta propuesta, se han desarrollado componentes de usuario que prueban la funcionalidad. Un primer componente genera un fichero cuyo descriptor es transferido al *kernel* para que este lo referencie, estableciendo así un canal de comunicación seguro. Por otro lado, la integridad de los datos reportados se confirma mediante un segundo componente de verificación que analiza la información escrita por el *kernel*, calculando localmente el HMAC del contenido con la misma clave secreta y contrastándolo posteriormente con el recibido para asegurar que coinciden y que el contenido es verídico.

Acrónimos

API *Application Programming Interface*

CIA *Confidentiality, Integrity and Availability*

CLI *Command Line Interface*

CPU *Central Processing Unit*

RAM *Random Access Memory*

FD *File Descriptor*

GNU *GNU's Not Unix*

OS *Operating System*

MAC *Message Authentication Code*

HMAC *Hash-based Message Authentication Code*

SHA *Symetric Hash Algorithm*

ID *Identifier*

LKM *Loadable Kernel Module*

PCB *Process Control Block*

US *User Space*

KS *Kernel Space*

NS *Namespace*

UID *User Identifier*

PID *Process Identifier*

GID *Group Identifier*

PS *Process Status*

VFS *Virtual File System*

FS *File System*

MMU *Memory Management Unit*

BSS *Block Started by Symbol*

PC *Program Counter*

SP *Stack Pointer*

SYS *System*

BUF *Buffer*

GFP *Get Free Pages*

Índice general

1. Introducción **1**
1.1. Sistemas operativos *Linux* 2
1.1.1. Arquitectura y anillos de privilegios 4
1.1.2. Memoria: Espacio de usuario y espacio de kernel 6
1.1.3. Sistema de Ficheros Virtual (*VFS*) 8
1.1.4. Espacios de nombres (*Namespaces*) 8
1.1.5. Programación y Módulos del Kernel de *Linux* (*LKMs*) 8
1.2. Ciberseguridad 8
1.3. Amenazas de seguridad 8
1.3.1. Números 9
1.3.2. Listas 9

2. Objetivos **11**
2.1. Descripción del problema 11
2.2. Requisitos 11
2.3. Competencias 11
2.4. Metodología 12
2.5. Plan de trabajo 12

3. Plataforma de desarrollo **13**

4. Diseño **14**
4.1. Snippets 14
4.2. Verbatim 14
4.3. Ecuaciones 15
4.4. Tablas o cuadros 15

5. Conclusiones **17**
5.1. Conclusiones 17
5.2. Corrector ortográfico 18

Bibliografía **19**

Índice de figuras

1.1. Árbol genealógico de los sistemas operativos *Unix* [1]. 3
1.2. Anillos de privilegios en sistemas operativos modernos [2, 3]. 4
1.3. Estructura de un sistema operativo moderno [3]. 5
1.4. Gestión de memoria virtual para procesos en *Linux* [3]. 7
1.5. División lógica de memoria en Espacio de Usuario y Espacio de Kernel [3]. 8
1.6. Robot aspirador Roomba de iRobot. 9

Listado de códigos

4.1. Función para buscar elementos 3D en la imagen 14
4.2. Cómo usar un Slider 15

Listado de ecuaciones

4.1. Ejemplo de ecuación con fracciones 15
4.2. Ejemplo de ecuación con array y letras y símbolos especiales 15

Índice de tablas

4.1. Parámetros intrínsecos de la cámara 16

Capítulo 1
Introducción

«*El mayor enemigo de la seguridad es la complejidad.*»

Bruce Schneier *(A Plea for Simplicity, 1999)* [4].

Hoy en día, la seguridad de los sistemas es un aspecto fundamental a tener en cuenta, cobrando cada vez más importancia a medida que alojamos mayor cantidad de información sensible en ellos y se convierten en sistemas mucho más complejos. Por lo tanto, podrían quedar expuestos a un gran número de amenazas si no se protegen debidamente. Como advierte el ensayo mostrado al inicio del capítulo, la inherente complejidad de estas infraestructuras facilita la proliferación de *malware* diseñado para operar e infiltrarse desde la más estricta invisibilidad.

Un sistema operativo moderno no es una estructura o proceso simple, sino un entrelazado conjunto de capas o niveles concéntricos de abstracción que actúan respecto a un sistema de hardware. Se puede describir como un sistema realmente complejo, con una gran cantidad de componentes y funcionalidades. En consecuencia, esta complejidad hace que sea difícil de proteger y monitorizar, requiriendo de un gran conocimiento sobre el mismo para saber cómo actuar ante posibles ataques.

A su vez, por parte del atacante, se requiere de un gran conocimiento técnico para diseñar un ataque efectivo, pero también de una gran creatividad y habilidad para encontrar vulnerabilidades y las formas de explotarlas sin ser detectado, dependiendo de su finalidad.

Esto hace de la ciberseguridad una disciplina realmente amplia y compleja, donde atacantes y defensores se encuentran en una constante carrera por encontrar nuevas formas de comprometer o proteger los sistemas.

Este proyecto se centra en el ámbito de la seguridad de los sistemas operativos *Linux*, concretamente en la protección ante amenazas que se infiltran en el núcleo del sistema, como los *rootkits*, evitando la manipulación de información sensible hacia los usuarios mediante un canal de reporte seguro y verificable.

Para comprender en detalle el escenario descrito y contextualizar el proyecto en sí, se explicarán en las siguientes secciones los fundamentos teóricos pertinentes. A continuación, se analizará la arquitectura y estructura de los sistemas operativos, principalmente *Linux*, sistema utilizado en el proyecto. Posteriormente, se definirá más en detalle en qué se centra la ciberseguridad y sus finalidades. Finalmente, se profundizará en qué consisten las principales amenazas que comprometen estos entornos.

1.1. Sistemas operativos *Linux*

No existe una definición canónica y universal para el concepto de sistema operativo, tratándose de una idea hasta cierto punto relativa y subjetiva dependiendo de su cometido y contexto de uso [3].

Desde un punto de vista físico, puede definirse como el conjunto de programas que permite utilizar la máquina como sistema, haciendo lo necesario para que puedan ejecutarse múltiples procesos sobre el mismo *hardware*, multiplexando la máquina en tiempo (administrando uso de *CPU*) y espacio (gestionando la memoria *RAM*).

Por otro lado, desde un punto de vista lógico, se puede definir como el programa o conjunto de los mismos que actúan como intermediarios entre el usuario y el *hardware*, proporcionando una interfaz como abstracción de procesos e instrucciones internas, para poder interactuar con la máquina y sus recursos de manera eficiente y robusta.

En el mercado tecnológico actual existen diferentes sistemas operativos de propósito general, destacando opciones comerciales de código cerrado como *Microsoft Windows* o *macOS*, orientados generalmente a ordenadores, y por otra parte, *Android* e *iOS*, principalmente diseñados para dispositivos móviles. Sin embargo, dentro del ámbito de la administración de infraestructuras, ciberseguridad y desarrollo de *software*, *GNU/Linux* (de la familia *Unix*) se considera como el estándar de referencia absoluto.

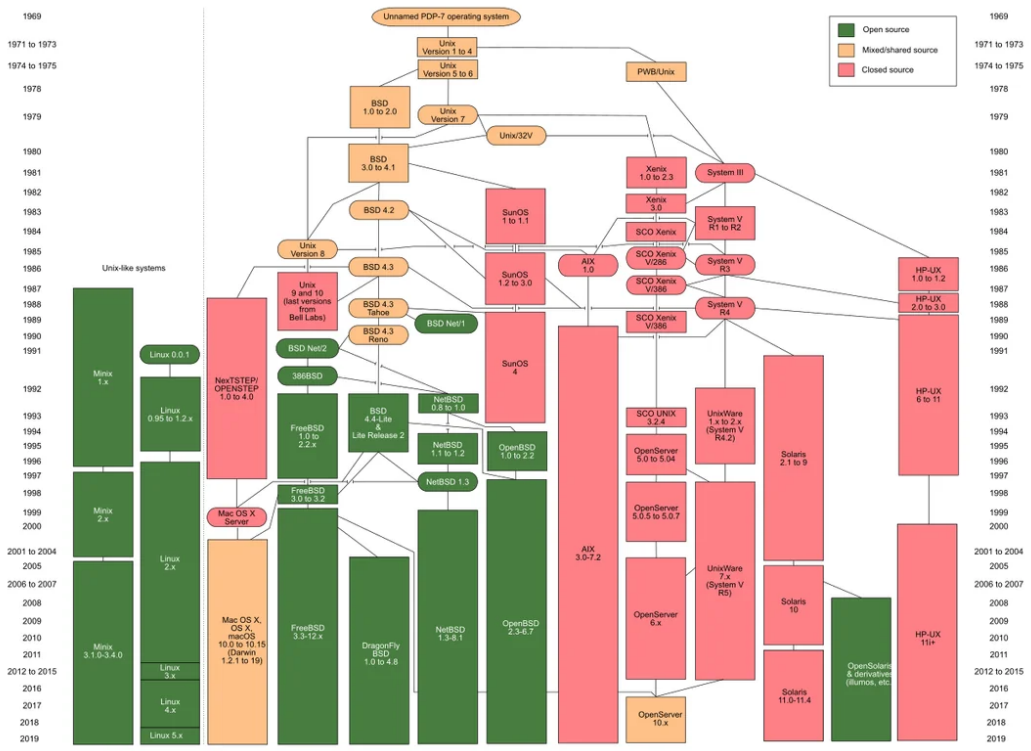

Figura 1.1: Árbol genealógico de los sistemas operativos *Unix* [1].

Como ilustra la imagen anterior, *Unix* es el sistema pionero creado en los años 70 que sentó las bases de la informática e inspiró directamente el desarrollo de *Linux*, considerado como su evolución natural de código abierto. También *macOS* proviene de esta familia; sin embargo, este sistema operativo ha evolucionado bajo un modelo comercial semi-privativo y de código parcialmente cerrado.

La hegemonía de *GNU/Linux* en estos sectores se debe principalmente a su filosofía de código abierto (*open-source*). Disponer del código fuente de manera pública, tanto directamente en los directorios del sistema (típicamente en `/usr/src`) como en repositorios públicos en línea como *GitHub*, permite a una gran comunidad mundial de desarrolladores auditar, depurar y mejorar el código del mismo de forma continua. Esta transparencia garantiza un nivel de adaptabilidad y fiabilidad muy superior al de los sistemas comerciales, permitiendo una configuración del entorno a medida según las necesidades requeridas.

Por estos motivos, se considera *GNU/Linux* el sistema operativo óptimo para desarrollar este proyecto. Por consiguiente, a partir de este punto se asumirá por defecto que nos encontramos en un entorno *Linux*.

1.1.1. Arquitectura y anillos de privilegios

Para garantizar su estabilidad y seguridad, los sistemas operativos modernos no confían únicamente en restricciones lógicas de *software*, sino que se apoyan de forma entrelazada en mecanismos de protección implementados a nivel de *hardware* por el propio procesador. En arquitecturas de sistemas predominantes como la familia x86, este mecanismo de aislamiento se implementa típicamente a través de dominios concéntricos de protección jerárquicos, comúnmente conocidos como anillos de protección (*Protection Rings*) [3].

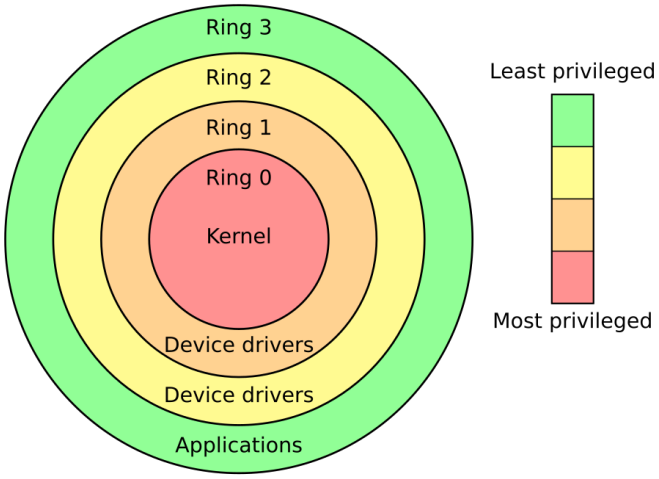

Figura 1.2: Anillos de privilegios en sistemas operativos modernos [2, 3].

Esta arquitectura determina principalmente cuatro niveles de privilegios concéntricos, comúnmente llamados "anillos", numerados del 0 al 3. El nivel de privilegio implica qué instrucciones de la CPU puede ejecutar un proceso y a qué regiones de la memoria puede acceder al ejecutarse, distinguiendo estos cuatro anillos [3]:

- ***Ring 0*:** Nivel de máximo privilegio, donde reside típicamente el núcleo (*kernel*). Los programas ejecutados en este anillo tienen prácticamente acceso directo e irrestricto a todo el *hardware*, la memoria y las instrucciones del procesador.

- ***Ring 1 y 2*:** Originalmente diseñados para servicio y control del propio sistema con privilegios intermedios, aunque en la práctica han sido mayormente descartados por la mayoría de sistemas operativos, debido a su complejidad y bajo rendimiento.

- ***Ring 3*:** Nivel de menor privilegio. Los procesos lanzados en este anillo no pueden interactuar directamente con el *hardware*, debiendo delegar cualquier operación crítica a niveles de mayor privilegio.

Por motivos de eficiencia y portabilidad a otras arquitecturas de procesadores (como *ARM*, que normalmente no posee tantos niveles), *Linux* simplifica este modelo y utiliza exclusivamente dos de estos anillos. Por un lado, se aísla el *kernel* (núcleo del sistema), ejecutándolo íntegramente en el *Ring 0*, y por otro lado, se confina el resto de programas de usuario en la región del *Ring 3* [5, 3].

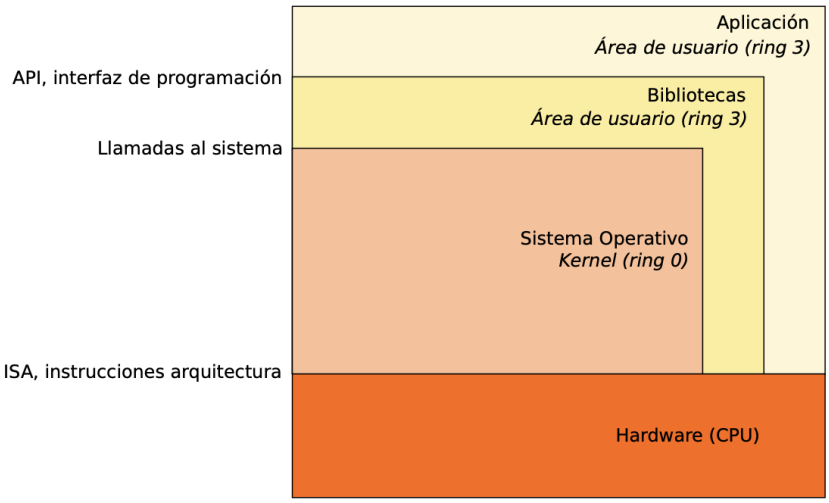

Figura 1.3: Estructura de un sistema operativo moderno [3].

Básicamente, la estructura del sistema operativo se puede describir como una división física de privilegios que se traduce directamente en una arquitectura lógica estructurada en capas construidas unas encima de otras [3].

En la base se encuentra el *hardware* físico de la máquina, con el cual el núcleo interactúa directamente a través de las instrucciones de la arquitectura (ISA) disponibles. Inmediatamente por encima, operando con privilegios absolutos sobre el sistema, en el *Ring 0*, reside el sistema operativo (*kernel*). Posteriormente, en las capas más externas del mismo, se encuentran las bibliotecas del sistema, encargadas de proporcionar la ***interfaz de programación de aplicaciones (API)***, y las aplicaciones finales, las cuales se encuentran confinadas en el área de usuario (*Ring 3*).

Debido a esta separación, si algún proceso de esta región de usuario requiere realizar una acción privilegiada (como leer un archivo del disco, enviar un paquete de red o reservar memoria), no puede acceder al *hardware* directamente por sí mismo. En su lugar, debe solicitar dicha operación privilegiada al *kernel* atravesando la frontera del *Ring 0* mediante un mecanismo de comunicación estandarizado y controlado, comúnmente conocido como ***llamadas al sistema*** (*System Calls* o *syscalls*).

Esta barrera arquitectónica no solo define qué instrucciones se pueden ejecutar y dónde, sino que obliga al sistema operativo a dividir la memoria principal de rápido acceso (*RAM*) en dos grandes regiones virtuales completamente aisladas en ***espacio de usuario*** y ***espacio de kernel***, garantizando así controlar que ningún proceso de usuario pueda corromper datos sensibles internos del sistema [5, 3].

1.1.2. Memoria: Espacio de usuario y espacio de kernel

Para comprender cómo se materializa la división de privilegios descrita en el apartado anterior realmente en la memoria física de la máquina (*RAM*), es necesario introducir el concepto de ***memoria virtual***.

En los sistemas *Linux* modernos, los procesos no acceden directamente a las direcciones físicas reales de la memoria *RAM*. En su lugar, la *CPU*, a través de su unidad de gestión de memoria *MMU* y el *kernel*, colaboran para proporcionar a cada proceso la ilusión de que posee un espacio de direcciones de memoria contiguo, continuo y exclusivo.

Para lograr esta abstracción, el sistema divide las direcciones de memoria virtual del proceso en bloques de tamaño fijo llamados ***páginas***, mientras que la memoria física se divide en bloques reales equivalentes denominados ***marcos***. Cuando un proceso en ejecución requiere acceder a sus instrucciones (*Text*), a sus variables globales inicializadas (*Data*) o sin inicializar (*BSS*), a la memoria asignada dinámicamente (*Heap*) o a las variables locales de su *Pila* (*Stack*), la *MMU* consulta una ***tabla de páginas*** para traducir dinámicamente esa dirección virtual de memoria a su ubicación física real. De este modo, múltiples procesos pueden ejecutarse simultáneamente de forma segura sin correr ningún riesgo para el sistema, ya que cada uno cuenta con su propia estructura de ***memoria virtual*** y su tabla de mapeo aislada, gestionada íntegramente por el sistema operativo (*kernel*) [6].

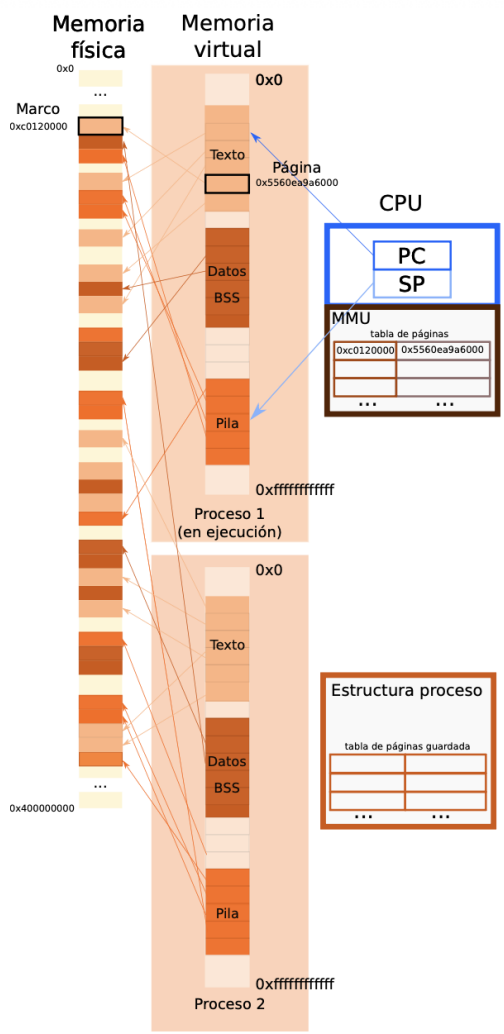

Figura 1.4: Gestión de memoria virtual para procesos en *Linux* [3].

A su vez, este espacio de *memoria virtual* se divide de forma estricta en dos grandes regiones, fuertemente relacionadas con la arquitectura de anillos de protección del procesador [6]:

- **Espacio de Usuario (*User Space*):** Es la porción inferior de la memoria virtual, asociada al *Ring 3*. En esta región se cargan y ejecutan las aplicaciones de usuario, sus bibliotecas y los datos pertinentes. Si un proceso comete un error "crítico" en esta región (como intentar acceder a memoria que no le pertenece), el sistema operativo interviene para finalizar dicho proceso y genera un error (típicamente conocido como *Segmentation Fault*), evitando que el fallo afecte al resto del sistema.

- **Espacio del Kernel (*Kernel Space*):** Es la porción superior de la memoria virtual, reservada exclusivamente para el ***kernel*** con sus módulos y los

controladores de dispositivos (*drivers*). Al estar asociado al *Ring 0*, el código que reside aquí tiene acceso total a la memoria física y, por consiguiente, al *hardware*. Un fallo de programación o una intrusión maliciosa en este nivel compromete todo el sistema, pudiendo provocar una caída total del mismo (*Kernel Panic*).

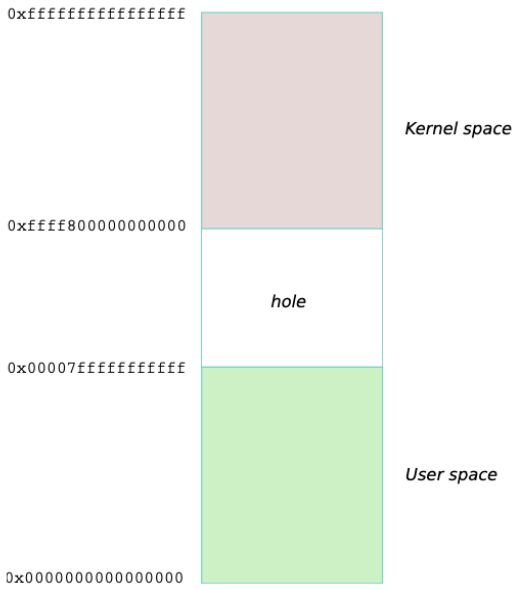

Figura 1.5: División lógica de memoria en Espacio de Usuario y Espacio de Kernel [3].

De la misma manera que la *memoria virtual* abstrae la complejidad de organización de la memoria física (*RAM*) para garantizar el aislamiento y la ejecución segura de los procesos, el sistema operativo proporciona a su vez una abstracción equivalente para la gestión del almacenamiento persistente y la interacción con el resto de componentes del sistema. Esta importante tarea recae sobre otro gran pilar estructural de un sistema operativo, en este caso *Linux*: su *sistema de ficheros* (*file system*).

1.1.3. Sistema de Ficheros Virtual (*VFS*)

1.1.4. Espacios de nombres (*Namespaces*)

1.1.5. Programación y Módulos del Kernel de *Linux* (*LKMs*)

1.2. Ciberseguridad

1.3. Amenazas de seguridad

En los textos puedes poner palabras en *cursiva*, para aquellas expresiones en sentido *figurado*, palabras como *robota*, que está fuera del diccionario castellano, o bien para resaltar palabras de una colección: *(a)* es la primera letra del abecedario, *(b)* es la

segunda, etc.

Al poner las dos líneas del anterior párrafo, este aparecerá separado del anterior. Si no las pongo, los párrafos aparecerán pegados. Sigue el criterio que consideres más oportuno.

No olvides incluir imágenes y referenciarlas, como la Figura 1.6.

Figura 1.6: Robot aspirador Roomba de iRobot.

Ni tampoco olvides de poner las URLs como notas al pie. Por ejemplo, si hablo de la Robocup[1].

1.3.1. Números

En lugar de tener secciones interminables, como la Sección **??**, divídelas en subsecciones.

Para hablar de números, mételos en el entorno *math* de LaTeX, por ejemplo, $1{,}5Kg$. También puedes usar el símbolo del Euro como aquí: 1.500€.

1.3.2. Listas

Cuando describas una colección, usa `itemize` para ítems o `enumerate` para enumerados. Por ejemplo:

- *Entorno de simulación.* Hemos usado dos entornos de simulación: uno en 3D y otro en 2D.
- *Entornos reales.* Dentro del campus, hemos realizado experimentos en Biblioteca y en el edificio de Gestión.

[1] `http://www.robocup.org`

1. Primer elemento de la colección.
2. Segundo elemento de la colección.

Referencias bibliográficas Cita, sobre todo en este capítulo, referencias bibliográficas que respalden tu argumento. Para citarlas basta con poner la instrucción `\cite` con el identificador de la cita. Por ejemplo: libros como [7], artículos como [8], URLs como [9], tesis como [10], congresos como [11], u otros trabajos fin de grado como [12].

Las referencias, con todo su contenido, están recogidas en el fichero `bibliografia.bib`. El contenido de estas referencias está en formato `BibTex`. Este formato se puede obtener en muchas ocasiones directamente, desde plataformas como `Google Scholar` u otros repositorios de recursos científicos.

Existen numerosos estilos para reflejar una referencia bibliográfica. El estilo establecido por defecto en este documento es APA, que es uno de los estilos más comunes, pero lo puedes modificar en el archivo `memoria.tex`; concretamente, cambiando el campo `apalike` a otro en la instrucción `\bibliographystyle{apalike}`.

Y, para terminar este capítulo, resume brevemente qué vas a contar en los siguientes.

Capítulo 2
Objetivos

Quizás algún fragmento de libro inspirador...

Autor, *Título*

Escribe aquí un párrafo explicando brevemente lo que vas a contar en este capítulo. En este capítulo lo ideal es explicar cuáles han sido los objetivos que te has fijado conseguir con tu trabajo, qué requisitos ha de respetar el resultado final, y cómo lo has llevado a cabo; esto es, cuál ha sido tu plan de trabajo.

2.1. Descripción del problema

Cuenta aquí el objetivo u objetivos generales y, a continuación, concrétalos mediante objetivos específicos.

2.2. Requisitos

Describe los requisitos que ha de cumplir tu trabajo.

2.3. Competencias

Enumera las competencias generales adquiridas y empleadas. Es decir, las que consideres que has adquirido durante la realización de tu TFG (las podrás encontrar en la guía docente de la asignatura TFG), así como las que creas que has adquirido en las distintas asignaturas del grado (descritas en sus respectivas guías docentes) y que has empleado para llevarlo a cabo.

2.4. Metodología

Qué paradigma de desarrollo software has seguido para alcanzar tus objetivos.

2.5. Plan de trabajo

Qué agenda has seguido. Si has ido manteniendo reuniones semanales, cumplimentando objetivos parciales, si has ido afinando poco a poco un producto final completo, etc.

Capítulo 3
Plataforma de desarrollo

Quizás algún fragmento de libro inspirador...

Autor, *Título*

Escribe aquí un párrafo explicando brevemente lo que vas a contar en este capítulo. En este capítulo, explica qué has usado a nivel hardware y software para poder desarrollar tu trabajo: librerías, sistemas operativos, plataformas, entornos de desarrollo, etc.

Capítulo 4
Diseño

Quizás algún fragmento de libro inspirador...

Autor, *Título*

Escribe aquí un párrafo explicando brevemente lo que vas a contar en este capítulo. En este capítulo (y quizás alguno más) es donde, por fin, describes detalladamente qué has hecho y qué experimentos has llevado a cabo para validar tus desarrollos.

4.1. Snippets

Puede resultar interesante, para clarificar la descripción, mostrar fragmentos de código (o *snippets*) ilustrativos. En el Código 4.1 vemos un ejemplo escrito en `C++`.

```
void Memory::hypothesizeParallelograms () {
  for(it1 = this->controller->segmentMemory.begin(); it1++) {
    squareFound = false; it2 = it1; it2++;
    while ((it2 != this->controller->segmentMemory.end()) && (!squareFound))
        {
      if (geometry::haveACommonVertex((*it1),(*it2),&square)) {
        dist1 = geometry::distanceBetweenPoints3D ((*it1).start, (*it1).end);
        dist2 = geometry::distanceBetweenPoints3D ((*it2).start, (*it2).end);
      }
    // [...]
```

Código 4.1: Función para buscar elementos 3D en la imagen

En el Código 4.2 vemos un ejemplo escrito en `Python`.

4.2. Verbatim

Para mencionar identificadores usados en el código —como nombres de funciones o variables— en el texto, usa el entorno literal o verbatim

```
def mostrarValores():
    print (w1.get(), w2.get())

master = Tk()
w1 = Scale(master, from_=0, to=42)
w1.pack()
w2 = Scale(master, from_=0, to=200, orient=HORIZONTAL)
w2.pack()
Button(master, text='Show', command=mostrarValores).pack()

mainloop()
```

Código 4.2: Cómo usar un Slider

`hypothesizeParallelograms()`. También se puede usar este entorno para varias líneas, como se ve a continuación:

```
void Memory::hypothesizeParallelograms () {
  // add your code here
}
```

4.3. Ecuaciones

Si necesitas insertar alguna ecuación, puedes hacerlo. Al igual que las figuras, no te olvides de referenciarlas. A continuación se exponen algunas ecuaciones de ejemplo: Ecuación 4.1 y Ecuación 4.2.

$$H = 1 - \frac{\sum_{i=0}^{N} \frac{(\frac{d_{js}+d_{je}}{2})}{N}}{M} \tag{4.1}$$

Ecuación 4.1: Ejemplo de ecuación con fracciones

$$v(entrada) = \begin{cases} 0 & \text{if} \quad \epsilon_t < 0{,}1 \\ K_p \cdot (T_t - T) & \text{if} \quad 0{,}1 \leq \epsilon_t < M_t \\ K_p \cdot M_t & \text{if} \quad M_t < \epsilon_t \end{cases} \tag{4.2}$$

Ecuación 4.2: Ejemplo de ecuación con array y letras y símbolos especiales

4.4. Tablas o cuadros

Si necesitas insertar una tabla, hazlo dígnamente usando las propias tablas de LaTeX, no usando pantallazos e insertándolas como figuras... En el Cuadro 4.1 vemos

un ejemplo.

Parámetros	**Valores**
Tipo de sensor	Sony IMX219PQ[7] CMOS 8-Mpx
Tamaño del sensor	3.674 x 2.760 mm (1/4"format)
Número de pixels	3280 x 2464 (active pixels)
Tamaño de pixel	1.12 x 1.12 um
Lente	f=3.04 mm, f/2.0
Ángulo de visión	62.2 x 48.8 degrees
Lente SLR equivalente	29 mm

Tabla 4.1: Parámetros intrínsecos de la cámara

Capítulo 5
Conclusiones

Quizás algún fragmento de libro inspirador...

Autor, *Título*

Escribe aquí un párrafo explicando brevemente lo que vas a contar en este capítulo, que básicamente será una recapitulación de los problemas que has abordado, las soluciones que has prouesto, así como los experimentos llevados a cabo para validarlos. Y con esto, cierras la memoria.

5.1. Conclusiones

Enumera los objetivos y cómo los has cumplido.

Enumera también los requisitos implícitos en la consecución de esos objetivos, y cómo se han satisfecho.

No olvides dedicar un par de párrafos para hacer un balance global de qué has conseguido, y por qué es un avance respecto a lo que tenías inicialmente. Haz mención expresa de alguna limitación o peculiaridad de tu sistema y por qué es así. Y también, qué has aprendido desarrollando este trabajo.

Por último, añade otro par de párrafos de líneas futuras; esto es, cómo se puede continuar tu trabajo para abarcar una solución más amplia, o qué otras ramas de la investigación podrían seguirse partiendo de este trabajo, o cómo se podría mejorar para conseguir una aplicación real de este desarrollo (si es que no se ha llegado a conseguir).

5.2. Corrector ortográfico

Una vez tengas todo, no olvides pasar el corrector ortográfico de LaTeXa todos tus ficheros *.tex*. En `Windows`, el propio editor `TeXworks` incluye el corrector. En `Linux`, usa `aspell` ejecutando el siguiente comando en tu terminal:

```
aspell --lang=es --mode=tex check capitulo1.tex
```

Bibliografía

[1] G. Anthes, "Unix turns 40: The past, present and future of a revolutionary os," *Operative Systems, Linux*, June 2009. [Online]. Available: https://www.computerworld.com/article/1553843/unix-turns-40-the-past-present-and-future-of-a-revolutionary-os.html

[2] C. de Wikipedia, "Anillo (seguridad informática)," Wikipedia, online article, January 2026. [Online]. Available: https://es.wikipedia.org/wiki/Anillo_(seguridad_informtica)

[3] E. Soriano Salvador and G. Guardiola Múzquiz, *Fundamentos de Sistemas Operativos: Una aproximación práctica usando Linux*. GSyC, Universidad Rey Juan Carlos, February 2024. [Online]. Available: https://honecomp.github.io/librossoo.html

[4] B. Schneier, "A plea for simplicity," Schneier on Security, online essay, November 1999. [Online]. Available: https://www.schneier.com/essays/archives/1999/11/a_plea_for_simplicit.html

[5] R. Love, *Linux Kernel Development*, 3rd ed. Developer's Library, Addison-Wesley, 2010, ch. 1. [Online]. Available: https://github.com/swadhinsekhar/books/blob/master/Linux%20Kernel%20Development%203rd%20Edition%20-%20Love%20-%202010.pdf

[6] A. Silberschatz, P. B. Galvin, and G. Gagne, *Fundamentos de sistemas operativos*, 9th ed. Madrid, España: McGraw-Hill Interamericana, 2013, ch. 9. [Online]. Available: https://books.google.es/books?id=b2qynQEACAAJ

[7] J. Vega, E. Perdices, and J. Cañas, *Attentive visual memory for robot localization*. IGI Global, USA, September 2012, pp. 408–438, text not available. This book is protected by copyright.

[8] J. Vega and J. Cañas, "PyBoKids: An innovative python-based educational framework using real and simulated Arduino robots," *Electronics*, vol. 8, pp. 899–915, August 2019.

[9] J. Vega, "El profesor Julio Vega, finalista del concurso 'Ciencia en Acción 2019'," URJC, on-line newspaper interview, July 2019.

[10] ——, "Educational framework using robots with vision for constructivist teaching Robotics to pre-university students," Doctoral thesis on Computer Science and Artificial Intelligence, University of Alicante, September 2018.

[11] ——, "JdeRobot-Kids framework for teaching robotics and vision algorithms," in *II jornada de investigación doctoral*. University of Alicante, June 2018.

[12] ——, "Navegación y autolocalización de un robot guía de visitantes," Master thesis on Computer Science, Rey Juan Carlos University, September 2008.

[13] ——, "De la tiza al robot," Franciscanas Montpellier Schools, annual report, Tech. Rep., June 2015.